Deep Learning (CS60010)
Spring Semester 2026
Assignment 2: Open Architecture Design Challenge on ARC-AGI
Date: April 27, 2026

Important Instructions:

1. **Programming Language / Environment**
   - This assignment is to be completed within a Colab notebook.
   - You can use Python and a Deep Learning framework (PyTorch) compatible with the Colab T4 environment.
   - Code must be readable, well-commented, and use docstrings for non-obvious design choices.
2. **Hard Constraints & Parameter Budget**
   - The total parameter count for your model (including backbone weights, embeddings, and output heads) **must not exceed 50 Million (≤ 50M)**.
   - You must train your model from scratch, starting with randomly initialized weights.
   - Only the ARC-1 training data (400 tasks) may be used for development, training, and validation. No synthetic data, other datasets, or pretrained models are allowed.
3. **Input / Output / Evaluation**
   - The model must be able to process the ARC-AGI grid data format (JSON files).
   - A task is only considered solved if the predicted output grid exactly matches the target (exact-match accuracy).
   - A final evaluation must be run on the 400 evaluation tasks.
   - This is a team assignment (5 members in a team). There will be one submission per team, and in case of multiple submissions the latest submission will be taken into consideration.
4. **Submission**
   - You must submit a single **Jupyter Notebook** (run end-to-end in a fresh runtime). The naming convention should be **DL_Assignment_<team_name>. ipynb**.
   - The notebook must include a cell that prints the total parameter count.
   - The final submission must be a single ZIP file named **DL_Assignment_<team_name>.zip**. This zip file must contain three files: the Jupyter notebook of the best model (including ablation sections), the structured **Design Document Report DL_Assignment_<team_name>_report.pdf**, and a **README.md** file which explains how to run the proper cells.
   - Plagiarism in any form is strictly prohibited and will result in 0 marks for both the source and target assignments.

---

1. What is ARC-AGI 1?

Dataset: https://github.com/fchollet/ARC-AGI/tree/master

ARC-AGI 1 (Abstraction and Reasoning Corpus) is a benchmark of 800 visual reasoning puzzles designed to test a system's ability to learn new rules from minimal examples. Each puzzle presents small grids of coloured cells, integers 0 through 9 on a grid up to 30 × 30 and asks your model to identify an underlying rule from 2 to 4 demonstration pairs, then apply that rule to a new test input. Every puzzle uses a different rule, and no rules are written down anywhere. The model must discover the rule purely from the examples. There is no text, no natural language, no shared vocabulary of rules. This is a pure inductive reasoning task.

1.1 A concrete example

The following shows a real ARC-AGI task. Two demonstration pairs are shown. Your job is to figure out the transformation rule and apply it to the test input.

Pair	Input grid	Output grid
Demo 1	1 0 0 1 0 0 1 0 0	1 1 1 1 0 0 1 0 0
Demo 2	0 0 2 0 0 2 0 0 2	2 2 2 0 0 2 0 0 2
Test input	0 3 0 0 3 0 0 3 0	**??? (your model must predict)**

The rule here is: replicate the column containing the non-zero value across the entire top row. The correct answer for the test input is:

```
3 3 3
0 3 0
0 3 0
```

The rule is never written anywhere. It must be inferred from the two demonstrations alone.

1.2 The data format

Each task is a JSON file containing:

- A "train" field: a list of 2–4 input-output grid pairs (the demonstrations)
- A "test" field: one or more input grids whose outputs you must predict

Grids are stored as lists of lists, where each integer represents a colour (0 = black, 1–9 = specific colours). The smallest grid is 1×1; the largest is 30×30. Grid dimensions are not fixed, they can vary from task to task and even between input and output within the same task.

1.3 Evaluation

A task is counted as solved only if your predicted output grid exactly matches the target — every cell, every dimension. You receive 2 attempts per task. Your final score is the fraction of the 400 evaluation tasks solved by at least one of your two attempts. There is no partial credit for nearly-correct grids.

- data/training/ — 400 tasks. Use these for all training and validation.
- data/evaluation/ — 400 tasks. Use these only for final evaluation. Do not look at the outputs. Do not use these tasks for training or hyperparameter selection.

IMPORTANT: The evaluation task JSON files contain the test output grids in the file.

- Do not load or inspect them during development. Load only the inputs.
- If you accidentally train on evaluation labels, you will score 100% and learn nothing.
- The correct procedure: develop on training/, run your final model on evaluation/ inputs once, submit.

2. Your Task

Design, implement, train, and evaluate a model of your own invention that attempts to solve ARC-AGI 1 tasks. You are not given a prescribed architecture. You are not given a prescribed loss function. You are not given a prescribed training procedure.

Your job is to make deliberate design decisions across the axes described in Section 3, justify them, implement them carefully, measure their effect, and report your results honestly.

2.1 Hard constraints

Constraint	Detail
≤ 50M total parameters	Count everything: backbone weights, all embedding tables, output heads. Use a parameter counting utility and include the count in your notebook.

ARC-1 training data only	No synthetic data generation. No data from other datasets. No pretrained language models or vision models of any kind. You may augment the 400 training tasks as described in Section 3.
Train from scratch	No pretrained weights. All weights must be randomly initialised and trained only on your ARC-1 training data.
Evaluate on the public eval set	Report exact-match accuracy on data/evaluation/ with 2 attempts per task. Include a screenshot from the official evaluation.

2.2 What you are free to change

- Architecture: transformer, CNN, hybrid, recurrent, recursive, or anything else
- How you represent grids: flattened tokens, 2D tensors, coordinate pairs, one-hot cells
- Positional encoding: 1D, 2D, 3D, learned, RoPE, sinusoidal
- How you handle multiple demonstration pairs: concatenation, cross-attention, per-task embeddings, recurrence
- Loss function: token cross-entropy, cell-parallel prediction, latent-space losses, deep supervision, contrastive losses
- Training strategy: augmentation policy, test-time adaptation, curriculum, batch size, learning rate schedule, optimiser
- Inference strategy: greedy decoding, sampling, beam search, iterative refinement

3. The Design Space

This section is not a checklist of things to implement. It is a map of the decisions you will need to make. Read it carefully, pick the dimensions you think matter most, and go deep on those rather than trying to explore all of them.

3.1 How to represent a grid

A grid is a 2D array of integers 0–9. Your model cannot consume this directly — you must decide what it becomes. This is the most foundational decision you will make, and it directly affects what spatial reasoning is even possible.

Dimension	Options to explore	Key tradeoff
Flat token sequence	Flatten row by row into 1D. Each integer becomes a discrete token.	Simple to implement. Loses explicit spatial structure. A 30×30 grid = 900 tokens, very expensive.
Cell tokens with 2D position	Each cell is a token; its (row, col) is encoded separately via positional encoding.	Preserves spatial structure. Expensive for large grids but natural for attention.

One-hot colour vector per cell	Each cell becomes a 10-dim vector before any learned transformation.	Avoids treating colours as ordered integers. Adds a small parameter cost per layer.
Grid as 2D image tensor	Treat the grid as shape (H, W) and use CNN layers.	Strong spatial inductive bias. Output must also be a 2D grid, which needs a separate decoder design.
Run-length encoding	Compress consecutive same-value cells: '3 3 3 1' -> '3x3 1x1'.	Dramatically reduces sequence length. Loses exact position info within runs.

3.2 Positional encoding

This is one of the most consequential choices.. The reason: ARC rules are fundamentally spatial, they depend on which row a cell is in, which column, and which demonstration pair it belongs to.

Dimensions you can encode:

- Row index within a grid
- Column index within a grid
- Which demonstration pair (1st, 2nd, 3rd, or test)
- Whether the token is in an input or output grid

Encoding strategies:

- Learned absolute embeddings: simple, flexible, but require seeing all positions during training
- RoPE (rotary positional embeddings): can be extended to 2D or 3D, encodes relative distances naturally
- Sinusoidal: no parameters, but less expressive

Hint: 3D RoPE where each token has a (row, col, pair_index) position.

3.3 Handling multiple demonstration pairs

Each task provides 2–4 (input, output) pairs as context, plus a test input. The model must jointly condition on all of them to infer the rule.

The central challenge: the number of pairs varies per task, and there is no fixed ordering to the pairs (pair 1 is not semantically different from pair 2). Options:

- Concatenate all pairs into one sequence: straightforward, but a 3-pair task with 10×10 grids can exceed 600 tokens easily.
- Encode pairs independently and aggregate: encode each (input, output) pair with a shared encoder, then pool the resulting embeddings. Naturally handles variable-length pair sets.
- Per-task learnable embedding: give each training task a dedicated additive embedding. The model can learn task-specific features without storing them in the backbone weights. At evaluation time, either zero-initialise or retrieve a similar training task's embedding.
- Cross-attention between context and test input: encode the test input separately and use cross-attention to condition it on the demonstration pairs.
- Recurrent accumulation: process pairs one at a time, updating a hidden state. Natural fit for variable numbers of pairs.

3.4 Architecture choices

You are not required to use a transformer. Here are architectures that have been used or are worth exploring on ARC:

Transformer (decoder-only)

The standard autoregressive setup: flatten all context pairs and the test input into one sequence, train to predict output tokens causally. Simple baseline. Works surprisingly well with good positional encodings. **Main limitation:** autoregressive decoding of a 30×30 grid is 900 sequential steps.

Transformer (encoder-decoder)

Encode all demonstration pairs with a bidirectional encoder. Decode the output grid token by token conditioned on the encoder output. More parameters for the same backbone depth, but the encoder can look at the full context non-causally.

CNN-based

Treat each grid as a (H, W) image. A CNN processes each grid independently, producing spatial feature maps. The challenge is how to condition the test-input features on the demonstration-pair features. Options: concatenate feature maps, use FiLM conditioning, or use convolutional cross-attention.

Hybrid CNN + transformer

Use a CNN to extract local spatial features from each grid, then a small transformer to perform relational reasoning across cells and across pairs. The CNN handles local pattern extraction; the transformer handles global rule inference.

Recursive / iterative refinement

Start with an initial guess for the output grid (e.g. all zeros), then iteratively improve it by running the same network multiple times. Each pass takes the current guess plus the context as input and produces a refined guess. Key insight: this allows allocating more compute to hard tasks at inference time without changing the model size.

If you choose this approach, two implementation details matter:

- Deep supervision: compute loss at every refinement step, not just the final one. Recent analysis found this is the primary driver of performance gains, not the recursive structure itself.
- Latent state vs output state: maintain a separate latent variable z (reasoning state) and a separate y (current answer). The network updates both. This prevents z from spending capacity on tracking the answer format.

3.5 Loss functions

The most common choice is token-level cross-entropy on output grid cells. But there are alternatives worth considering, and you may combine multiple losses:

Loss function	How it works	When to consider it
Token cross-entropy(standard)	Predict each output cell token autoregressively. CE loss at each step.	Always a reasonable baseline. Well understood.
Parallel cell prediction	Predict all output cells simultaneously in one forward pass. No autoregressive dependency.	Faster training and inference. Loses inter-cell dependencies.
Deep supervision	Compute CE loss at every step T of a recursive/iterative model, sum them.	Essential if you use any form of iterative refinement. Significantly improves convergence.
Representation-space loss	Train a decoder to produce grid cells from a latent; minimise distance in latent space rather than token space.	Encourages compact intermediate representations. Higher implementation complexity.
Contrastive loss	Push representations of correct outputs close; push incorrect outputs away.	May help the model learn to discriminate valid from invalid completions.

3.6 Training strategy

You have 400 tasks. That is an extremely small dataset for any deep learning problem. Consider:

Data augmentation

Applying geometric transforms (flips, rotations) and colour permutations multiplies the effective training set size. A single task can become 8 variants under the dihedral group of the square, and many more under colour permutation.

Note: augmentation is not always safe. Colour permutation breaks tasks where the specific colour value is the rule (e.g. 'fill with colour 3 wherever colour 1 appears'). Before applying colour augmentation globally, consider whether to detect and exclude colour-identity tasks, or to apply colour augmentation only when it is semantically valid.

Test-time training (TTT)

This is legal and commonly used in top-performing approaches. After your model is trained, at evaluation time you briefly fine-tune it on the demonstration pairs of the specific task being tested without access to the test output. **The idea:** the demonstration pairs contain task-specific information your model did not see during training, and a few gradient steps can adapt the weights to that task.

TTT adds significant inference cost (training steps per task) but meaningfully improves accuracy. Whether the accuracy gain justifies the compute cost is an engineering decision you should measure and report.

Per-task embeddings

Give each of your 400 training tasks a dedicated learnable vector that is added to the input representation. During training, these vectors learn to encode task-specific structure. This is a form of meta-learning. The backbone learns to use the per-task hint, and the hints encode information the backbone cannot express generically.

At evaluation time for held-out tasks, the per-task embedding must be initialised somehow: use zeros, random, or retrieve the embedding from the most similar training task by some heuristic (grid statistics, colour count, size).

Optimiser and learning rate

Standard AdamW works. If you have time to experiment: recent small-model ARC work found NorMuon (a normalised variant of the Muon optimiser) converges faster and more stably than AdamW on small datasets. If you use AdamW, a WSD schedule (warmup then linear decay) tends to outperform pure cosine.

4. Compute Guidance

The T4 GPU on free Colab provides approximately 15 GB VRAM and runs for about 4 hours per session before disconnecting. You must design your workflow around this constraint.

4.1 Rough training time estimates

Model size	Context length	Approx. training time on T4
~5M active params	≤ 512 tokens	30–60 min for 400 tasks × 20 augments
~15M active params	≤ 512 tokens	1.5–3 hours
~30M active params	≤ 512 tokens	3–5 hours (2 sessions)
~50M active params	≤ 512 tokens	5–8 hours (3 sessions)

These are rough estimates, actual times depend heavily on your architecture, batch size, and how many augmented examples you generate. Budget conservatively. Always save checkpoints at the end of each session.

4.2 Practical workflow

1. Start with a small version of your model (~5M params) to validate that training works and losses are decreasing. Fix any bugs before scaling up.
2. Implement checkpoint saving and loading on Day 1. Use Google Drive mounting to persist checkpoints across sessions.
3. Run ablations in parallel with multiple accounts. Each ablation is a 1–2 hour run on its own account.
4. Keep sequence length ≤ 512 tokens per training example to stay within memory. If your serialisation produces longer sequences, use run-length encoding or a coarser representation.
5. Save your best checkpoint as checkpoint_best.pt and your final checkpoint as checkpoint_final.pt. Use the best checkpoint for evaluation.

4.3 Memory tips

- Use BF16 (brain float 16) training via torch.autocast. This roughly halves memory compared to FP32 at minimal accuracy cost.
- Use gradient checkpointing if your model is deep and you are hitting OOM errors (torch.utils.checkpoint.checkpoint_sequential).
- Use packed variable-length sequences rather than padded batches — this avoids wasting compute on padding tokens and significantly reduces memory.
- If running out of VRAM, reduce batch size and increase gradient accumulation steps to keep effective batch size constant.

5. What You Must Submit

5.1 Colab notebook

One notebook containing all your code: data loading, model definition, training loop, evaluation, and results. Requirements:

- Must run end-to-end in a fresh runtime
- Include a cell that prints the total parameter count using the utility above.
- Include cells showing your validation accuracy on a held-out subset of training tasks at different training stages.
- Include the final evaluation loop that generates predictions for the 400 evaluation tasks.
- Code must be readable. Use comments and docstrings for non-obvious design choices. A model definition that is a wall of uncommented code will receive partial marks.

5.2 Design document

A report structured exactly as follows.

Section	Content	Length
Section 1: Your approach	Describe your architecture, representation, positional encoding, loss function, and training procedure. For each major design choice, write one sentence justifying why you made it. End with a parameter count breakdown showing how the 50M budget is spent across backbone, embeddings, and heads.	~1.5 pages
Section 2: Ablations	Pick 3 design choices that you believe matter most. Train a version of your model with each choice replaced by a simpler baseline (e.g. 2D RoPE instead of 3D RoPE; no per-task embeddings; token CE instead of deep supervision). Present as a table: choice, your version, baseline version, accuracy difference. One sentence per row interpreting why the difference exists.	~1.5 pages
Section 3: Results	Your exact-match accuracy on the 400 evaluation tasks, with 2 attempts. Include the evaluation screenshot. One paragraph interpreting the result: which task categories did your model solve, which did it fail on, and why?	~0.5 pages
Section 4: What did not work	One paragraph describing something you tried and expected to work but did not. State your hypothesis about why it failed. This is assessed on honesty and reasoning, not on whether you eventually fixed it.	~1 pages

References:

1. https://arxiv.org/abs/2510.04871
2. https://arxiv.org/html/2512.06104
3. https://arxiv.org/html/2506.21734v1
4. https://mvakde.github.io/blog/why-all-ARC-solvers-fail-today/
5. https://mvakde.github.io/blog/44-on-arc-1/